

B2C D360 Personalization Plugin Setup Guide

Overview

This canvas documents the Business Manager setup for the **Data Cloud Personalization → SCAPI hydration** feature (`plugin_b2c_d360_personalization`). It covers the Cartridge Path, the 13 Custom Site Preferences, the Home/Cart Content Slots, and a browser-based verification walkthrough — so a developer can learn the whole setup at a glance instead of piecing it together from separate docs.

 **Verified against a live Business Manager instance** — Cartridge Path, all 13 Site Preferences, both Content Slot configs, and DevTools Console/Network verification are all captured below from a real, working setup, not mockups.

Why this exists

Data Cloud Personalization ingestion has real latency — commonly tens of minutes between a catalog or price update in B2C Commerce and that change reflecting in the Personalization decision index, since it has to flow through Data Cloud's batch ingestion pipeline. A storefront that renders tiles directly from Personalization-returned product data risks showing stale prices, sold-out items, or expired promotions in the meantime.

This cartridge avoids that by keeping Data Cloud off the rendering path entirely: Personalization only returns an array of recommended product IDs, and B2C Commerce hydrates each ID with real-time price, inventory, and promotion data — either through SCAPI Shopper Products (`scapi` mode) or the native `Tile-Show` controller (`sfra` mode). Time-sensitive data is always resolved live by B2C Commerce, never read from the Personalization payload itself.

Scope

This cartridge (`plugin_b2c_d360_personalization`) deploys:

- 1 SFRA controller (`PersonalizationTiles.js`) serving config, SLAS token, and price-lookup endpoints
- 1 script module (`configHelper.js`) reading the `ps_*` Site Preferences
- 1 plain ISML include + static client JS (`productRecommendations.isml` , `personalizationTiles.js`)
- 1 Page Designer custom component ("Personalization Product Recommendations")
- 1 Business Manager metadata extension (`system-objecttype-extensions.xml` , packaged as `ps-personalization-meta-import.zip`) defining the 13 `ps_*` Site Preferences

No database/job/service changes, and no dependency on other custom cartridges beyond the storefront base cartridge it sits ahead of on the Cartridge Path.

Environments

This canvas walks through the setup end-to-end on a single representative Business Manager instance. In a real rollout, the same setup repeats per environment with **environment-specific values only** — no code differs by environment:

Environment	Purpose	Config source
Development / Sandbox	Initial wiring, this canvas's walkthrough	Sandbox SCAPI org/short code, dev SLAS client
Staging	Customer UAT	Staging SCAPI org/short code, staging SLAS client, staging domain registered in SCAPI CORS Preferences
Production	Go-live	Production SCAPI org/short code, prod SLAS client, prod domain registered in SCAPI CORS Preferences

 **Never point a lower environment's SLAS client at production SCAPI data, or vice versa** — short code, org id, and client id must all belong to the same realm.

Recommended Rollout Sequence

1. **Sandbox — foundational setup.** Deploy the cartridge, set the Cartridge Path, import the metadata, configure the 13 Site Preferences, and wire `sfra` -mode zones into your storefront's existing product-tile markup (Parts 1–3).
2. **Staging / UAT — verification under realistic conditions.** Confirm `ps_pointNameMap` context resolution for every zone, walk the DevTools Console/Network

verification steps under real traffic (Part 4), and confirm activity-tracking telemetry reaches Data Cloud cleanly. If enabling `scapi` mode, this is also where you validate the staging SLAS client and its SCAPi CORS Preferences registration end-to-end (Part 5).

3. **Production — go-live.** Fold cartridge deployment into your standard release/CI-CD process, confirm the production SLAS client and its SCAPi CORS Preferences registration (if using `scapi` mode) and production Site Preference values, then go live.

Prerequisites

Before starting the setup below on any instance:

- ☐ Admin access to Business Manager for that instance (code deployment, Cartridge Path settings, Import & Export, and Page Designer if placing the component via Page Designer)
- ☐ Decided which render mode(s) you need for each zone — see "Choosing a Render Mode" below
- ☐ Confirmed the Decision Points you plan to reference in `ps_pointNameMap` / `ps_personalizationPointName` are already provisioned in Salesforce Personalization — this is done by the Data Cloud engineering team, not by this cartridge or Business Manager access (see Part 2 Q&A)
- ☐ If enabling activity tracking (`ps_activityTrackingEnabled`): confirmed the tenant-specific Data Cloud SDK Script URL (`ps_dcSdkUrl`), the `c360a.min.js` beacon) has been provisioned by Data Cloud for this tenant — tracking fails silently without a working URL here
- ☐ If enabling `scapi` mode: Account Manager access (or a contact who has it) to create/confirm a SLAS public (PKCE) client scoped to Shopper Products for that realm, **and** separate Account Manager access to register the storefront domain(s) against SCAPi CORS Preferences (a different admin API, and typically a different AM API client, than the SLAS client itself) — see Part 5
- ☐ Confirmed SCAPi short code, org id, and site id for that realm (only needed if enabling `scapi` mode)

Choosing a Render Mode

Both render modes ship in this cartridge and are production-ready. Which one to enable per zone depends on two things: whether your product tiles carry non-standard attributes, and your session/pricing requirements.

Question	Answer favors...
Do your product tiles rely on attributes SCAPi Shopper Products doesn't return — custom badges/overlays, swatch or size-indicator logic, pre-order flags, gender/category filtering, carousel-specific markup, etc.?	<code>sfra</code> — it reuses your native <code>Tile-Show</code> controller, so every custom attribute your storefront already renders elsewhere comes through with zero extra work.
Do you need customer-group pricing, active promotions, or logged-in-shopper personalization on the tile itself?	<code>sfra</code> — SCAPi Shopper Products only ever returns guest/list pricing (Part 4, Step 7).
Do you need guest-only pricing/availability fully decoupled from session state (e.g., for anonymous or cached rendering)?	<code>scapi</code> — it hydrates via a direct browser-to-SCAPi Shopper Products fetch, independent of session state.
Is minimizing new infrastructure (SLAS client, SCAPi CORS Preferences registration, Account Manager access) a priority for initial go-live?	<code>sfra</code> — no SLAS/SCAPi setup required at all.

If your tiles need attributes SCAPi doesn't return, `scapi` mode isn't unusable — it just needs custom enrichment logic added to backfill those fields from your own systems. For most implementations, starting with `sfra` mode and enabling `scapi` selectively (e.g., a specific guest-facing zone) is the lower-effort path.

Switching render mode per zone is a Business Manager-only change — no code deploy needed (Part 3).

Part 1: Code Deployment & Cartridge Path

Step 1 — Deploy the code. Upload the `plugin_b2c_d360_personalization` cartridge to the target instance (WebDAV, Business Manager Code upload, or the SFCC VS Code extension), then activate that code version in **Administration → Site Development → Code**. Without an active code version containing this cartridge, it won't appear in the Cartridge Path picker or the Page Designer component list.

■ **Optional: deploy via the B2C Commerce CLI instead of WebDAV/BM upload.** If your workflow uses the B2C Commerce CLI (`b2c`), the same Step 1 can be done with:

```
# Deploy all cartridges in the current project and activate the code version
b2c code deploy --code-version <version> --reload

# Deploy only specific cartridges
b2c code deploy --code-version <version> --cartridge plugin_b2c_d360_personalization
```

```
# Activate a previously uploaded code version
b2c code activate <version>

# Live-sync local file changes to the sandbox during development
b2c code watch --code-version <version>

# List code versions on the instance
b2c code list
```

See [Deploy and Iterate with the B2C Commerce CLI](#) for the full command reference.

Step 2 — Set the Cartridge Path.
Administration → Sites → Manage Sites → [Site] → Settings → Cartridges
Example cartridge path pattern:

```
plugin_b2c_d360_personalization:<your_custom_override_cartridge(s)>:app_storefront_base
```

⚠ **Order matters.** `plugin_b2c_d360_personalization` must sit before `app_storefront_base`, since it extends `PersonalizationTiles-*` controllers and templates. If you have your own custom override cartridge(s) that also override the `home / cart / PDP` templates where the personalization zones are embedded, those must appear before `app_storefront_base` too — otherwise those template overrides are silently ignored. As long as other unrelated custom cartridges don't also override the personalization templates, their position relative to this plugin doesn't matter functionally.

🖼 Screenshot: Cartridge settings page showing the path

Instance Type: All

→

Cartridges:

plugin_scapi_search:plugin_b2c_d360_personalization:app_storefront_po

Effective Cartridge Path:

plugin_scapi_search:plugin_b2c_d360_personalization:app_storefront_poc:app_storefront_base:plugin_apple_pay:plugin_facebook:plugin_payments:plugin_pinterest_commerce:plugin_web_payments:bc_content:core

Session Timeout:

30 Minutes

Part 2: Custom Site Preferences (13)

Merchant Tools → Site Preferences → Custom Preferences → Data Cloud Personalization
Imported via `cartridge/meta/system-objecttype-extensions.xml`. Import only creates the attributes/group — it does **not** set values.

Preference ID	Display Name	Type	Required?	Default
ps_enabled	Enable Personalization Tiles	Boolean	Optional	true
ps_scapiShortCode	SCAPI Short Code	String	Required	—
ps_scapiOrgId	SCAPI Organization ID	String	Required	—
ps_scapiClientId	SLAS Client ID	String	Required	—
ps_scapiSiteId	SCAPI Site ID	String	Required	—
ps_scapiRedirectUri	SLAS Redirect URI	String	Required	—
ps_personalizationPointName	Personalization Point Name	String	Optional	Generic_Product_Recommendations
ps_pointNameMap	Point Name Map (JSON)	Text	Optional	(empty)
ps_maxTiles	Max Tiles Per Zone	Integer	Optional	8
ps_expand	SCAPI Expand Params	String	Optional	availability, images, prices, promotions, vari
ps_dcSdkUrl	Data Cloud SDK Script URL	String	Optional	tenant beacon SDK URL

<code>ps_productIdField</code>	Personalization Product ID Field	String	Optional	<code>ssot__Id__c</code>
<code>ps_activityTrackingEnabled</code>	Enable Personalization Activity Tracking	Boolean	Optional	<code>true</code>

"Required" here means functionally required for `configHelper.isConfigured()` to return true — none are marked mandatory in the metadata itself, so BM will let you save with them blank; the feature just won't hydrate until all five are set.

`ps_dcSdkUrl` is tenant-specific too — it's the Data Cloud beacon script URL for this tenant, provisioned by Data Cloud, not something to fill in arbitrarily. Confirm it's been provisioned before enabling `ps_activityTrackingEnabled` (see Prerequisites).

Q&A: How `ps_pointNameMap` and `ps_personalizationPointName` interact

Q: What do these two preferences actually do? Every personalization zone (a Content Slot or Page Designer component) needs a Personalization "point name" — the identifier passed to the decision API to ask "what should I recommend here?" Rather than hardcoding one point name everywhere, each slot resolves its own via `configHelper.resolvePointName(architecture, contextKey, explicitName)`.

Q: What's the resolution order?

1. **Explicit override** — a point name entered directly on the slot/component instance, if set. Wins outright, skips the map entirely.
2. `ps_pointNameMap.architectures[<architecture>][<contextKey>]` — an architecture-specific entry (`sfra`, `pageDesigner`, or `pwa`) for that slot's `contextKey`.
3. `ps_pointNameMap.architectures.common[<contextKey>]` — the shared fallback entry, used when no architecture-specific override exists.
4. `ps_personalizationPointName` — the site-wide default, used only when the slot has no `contextKey` at all, or that `contextKey` isn't found anywhere in the map.

Q: What does the example map on this site actually resolve to?

contextKey	Architecture entry?	Resolves via	Point name
<code>home-hero</code>	none (sfra/pageDesigner/pwa empty)	<code>common</code>	<code>SK_Home</code>
<code>home-secondary</code>	none	<code>common</code>	<code>SK_Home</code>
<code>cart-crosssell</code>	none	<code>common</code>	<code>SK_Cart</code>
<code>cart-recently-viewed</code>	none	<code>common</code>	<code>SK_Cart</code>
(any other/no contextKey)	—	site default	<code>Generic_Product_Recommendations</code>

Q: Why have empty `sfra` / `pageDesigner` / `pwa` objects if nothing's in them yet? Different rendering paths could eventually need a different point name for the same logical placement — e.g. if a future PWA storefront should call a different Personalization zone than SFRA does for the same `home-hero` slot. The schema supports that per-architecture override today; until one is added, every architecture falls through to `common`.

Q: What happens if `ps_pointNameMap` is left blank or has malformed JSON? `configHelper.getPointNameMap()` catches the parse failure, logs a warning via `dw/system/Logger`, and returns an empty map — every slot then falls back to `ps_personalizationPointName` instead of breaking rendering.

Q: Where do `SK_Home` and `SK_Cart` themselves come from — are they defined in this cartridge? No — `SK_Home` and `SK_Cart` are **Decision Points** configured on the Data Cloud / Salesforce Personalization side, scoped per page context (home vs. cart). This cartridge's `ps_pointNameMap` only maps a storefront `contextKey` to whichever Decision Point name Data Cloud has already set up; it doesn't create or define those points. Provisioning a Decision Point is done by the **Data Cloud engineering team**, not by Business Manager access or this cartridge — adding a new slot on a new page context (e.g. a PDP zone) requires the Data Cloud engineers provisioning the corresponding Decision Point in Salesforce Personalization first, before a matching entry can be added to `ps_pointNameMap`.

Q: Where is `home-hero` actually defined — is it something a client sets in Business Manager? No — for Content Slots, `contextKey` is hardcoded in code, not a Business Manager field. Content Slot configuration in BM has no generic "Custom Attributes" screen, so each slot instead points its **Template** field at one of this cartridge's small dedicated template files (e.g. `slots/personalization/home.isml`), and that file's first line hardcodes the value: `<isset name="contextKey" value="{ 'home-hero' }" scope="pdict" />`. The template then delegates to the shared `productRecommendations.isml` include, which does the actual fetch/render generically for whatever `contextKey` it's handed.

Q: So what does a client actually need to do to add a new Content Slot with its own contextKey?

1. Create a new small template file that hardcodes the new `contextKey` (copy an existing one like `home.isml` and change the `isset` value) — this is the one step that requires a code change/deploy.
2. In Business Manager, create a new Content Slot (**Merchant Tools → Online Marketing → Content Slots**) and set its Template field to that new file.
3. Add an `<isslot id="...">` placeholder for that new slot ID in the page's ISML template, in the site's override cartridge (e.g. `app_storefront_poc`), not in `app_storefront_base` — also a code change.

4. Assign/activate the slot for the relevant page/context, same as the existing Home/Cart slots.
5. Optionally, add a `ps_pointNameMap` entry resolving the new contextKey to a Personalization point name in Business Manager (Site Preferences) — no code change needed for this step, just a preference edit.

Q: Is there a way to add a new personalization zone without any code change at all? Yes — use the **Page Designer component** instead of a Content Slot (see "Alternative: Page Designer placement" below). There, **Context Key** is a directly editable attribute on the component instance in Business Manager, so a business user can drop the component onto any page/region and set its own `contextKey` (plus `Render Mode`) with zero template changes or deploys.

 *Screenshot: Data Cloud Personalization preferences page showing all 13 fields*

Merchant Tools / Site Preferences / Custom Site Preference Groups /

Data Cloud Personalization ?

Instance Type

Sandbox

Search by ID

Q

T

Name	Value
Enable Personalization Tiles ← (ps_enabled) Enable or disable the personalization-to-SCAPI tile hydration feature on the storefront	Yes Enable or disable the personalization-to-SCAPI tile hydration feature...
SCAPI Short Code (ps_scapiShortCode) (String) The Commerce API short code for this org (e.g. "kv7kzm78"). Required.	The Commerce API short code for this org (e.g. "kv7kzm78"). Required...
SCAPI Organization ID (ps_scapiOrgId) (String) Salesforce Commerce API organization id (e.g. "f_ecom_zzrf_001"). Required.	f_ecom_zzbf_001 Salesforce Commerce API organization id (e.g. "f_ecom_zzrf_001"). ...
SLAS Client ID (ps_scapiClientId) (String) The public (PKCE) SLAS client id used to obtain the guest/shopper token for client-side SCAPI calls. Required. Its Account Manager CORS allowed-origins list must include this storefront's hostnam...	The public (PKCE) SLAS client id used to obtain the guest/shopper t...

SCAPI Site ID

(ps_scapiSiteId)
(String)

The SCAPI site id to scope product lookups to (e.g. "RefArchGlobal").

Required: ☐

RefArch

The SCAPI site id to scope product lookups to (e.g. "RefArchGlobal")...

SLAS Redirect URI

(ps_scapiRedirectUri)
(String)

A redirect_uri registered on the ps_scapiClientId SLAS client in Account Manager/SLAS Admin. Used server-side only for the guest PKCE token exchange (PersonalizationTiles->getToken) – SLAS never actual...

http://127.0.0.1:3000/callback

A redirect_uri registered on the ps_scapiClientId SLAS client in Acco...

Personalization Point Name

(ps_personalizationPointName)
(String)

Default Salesforce Personalization point/decision name to request when a slot (plain include or Page Designer component instance) doesn't specify its own point name and no entry matches it in the...

Generic_Product_Recommendations

Default Salesforce Personalization point/decision name to request ...

Point Name Map (JSON)

(ps_pointNameMap)
Optional JSON map resolving a slot's contextKey to a personalization point name, without editing code/Page Designer. Shape:
← version":1,"architectures":
{"common":{"&...

```
{
  "version": 1,
  "architectures": {
    "common": {
      "home-hero": "SK_Home",
      "home-secondary": "SK_Home",
      "cart-crosssell": "SK_Cart",
      "cart-recently-viewed": "SK_Cart"
    },
    "sfra": {}
  }
}
```

Optional JSON map resolving a slot's contextKey to a personalizatio...

Max Tiles Per Zone

(ps_maxTiles)
(Integer)

Maximum number of product tiles to render per zone. Also capped by SCAPI's 24-id-per-call batch limit. Default: 8.

Maximum number of product tiles to render per zone. Also capped ...

SCAPI Expand Params

(ps_expand)
(String)
Comma-separated SCAPI Shopper Products "expand" parameter values.
Default:
"availability,images,prices,promotion...

Comma-separated SCAPI Shopper Products "expand" parameter val...

Data Cloud SDK Script URL

(ps_dcSdkUrl)
(String)
Tenant-specific Salesforce Interactions/Data Cloud beacon SDK script URL (exposes window.getSalesforceInteractions()), e.g. "https://cdn.c360a.salesforce.com/...guid>/scripts...

https://cdn.c360a.salesforce.com/beacon/c360a/e5d520ec-c39

Tenant-specific Salesforce Interactions/Data Cloud beacon SDK scri...

(ps_productIdField)
(String)
Field name on each Personalization decision item that holds the product id. Default: "ssot__Id__c" – varies by customer implementation.

ssot__Id__c

Field name on each Personalization decision item that holds the pro...

Enable Personalization Activity Tracking

(ps_activityTrackingEnabled)
Enable or disable sending "personalization-view" and "personalization-click" activity events (via getSalesforceInteractions().sendEve... for tiles rendered by this feature. Requi...

Yes

Enable or disable sending "personalization-view" and "personalizati...

Part 3: Content Slots

Merchant Tools → Online Marketing → Content Slots

Slot ID	Page	Template (scapi mode)	Template (sfrs mode)	Purpose
home-personalization-m	Home	slots/personalization/home	slots/personalization/home-sfra	Primary homepage recommendation zone
home-personalization-scapi-m	Home	slots/personalization/home-scapi-compare	—	Side-by-side SCAPI cc column next to the pri
cart-personalization-m	Cart	slots/personalization/cart	slots/personalization/cart-sfra	Cart-page recommend

Switching a slot's **Template** field between the two options toggles render mode: **scapi** = client-side SLAS PKCE + SCAPI Shopper Products hydration; **sfra** = native **Title-Show** per product id (no SLAS/SCAPI setup, gets logged-in pricing for free). Save + re-Assign/activate after changing.

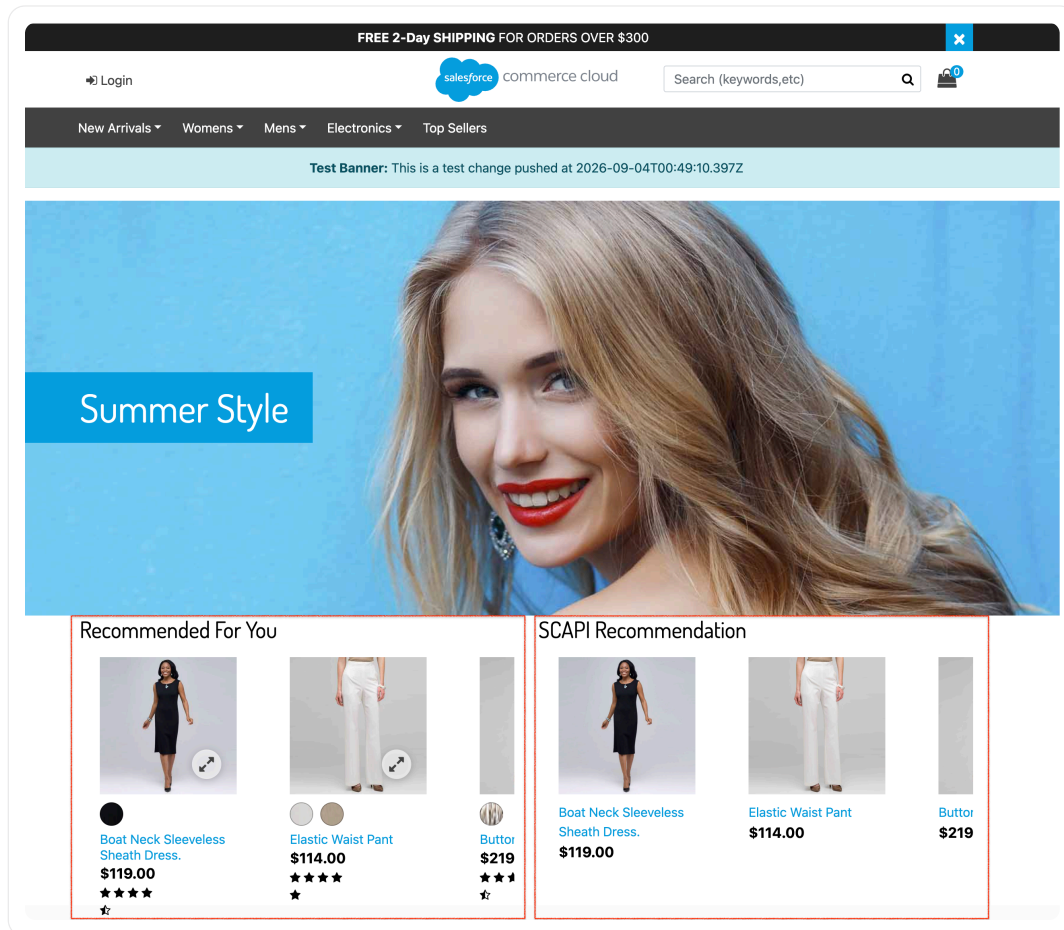
The cartridge also ships a matching template pair for PDP (**slots/personalization/pdp** / **slots/personalization/pdp-sfra**), plus a **home-scapi-compare** template for side-by-side SCAPI/SFRA testing. This walkthrough's screenshots cover Home and Cart; wiring a PDP zone follows the identical pattern — create a Content Slot on the PDP page (or drop the Page Designer component onto the PDP template) and pick the template/Render Mode matching your chosen hydration approach.

 Screenshot: Home slot config showing the Template field and both options

Template: * slots/personalization/home

Template: * slots/personalization/home-sfra

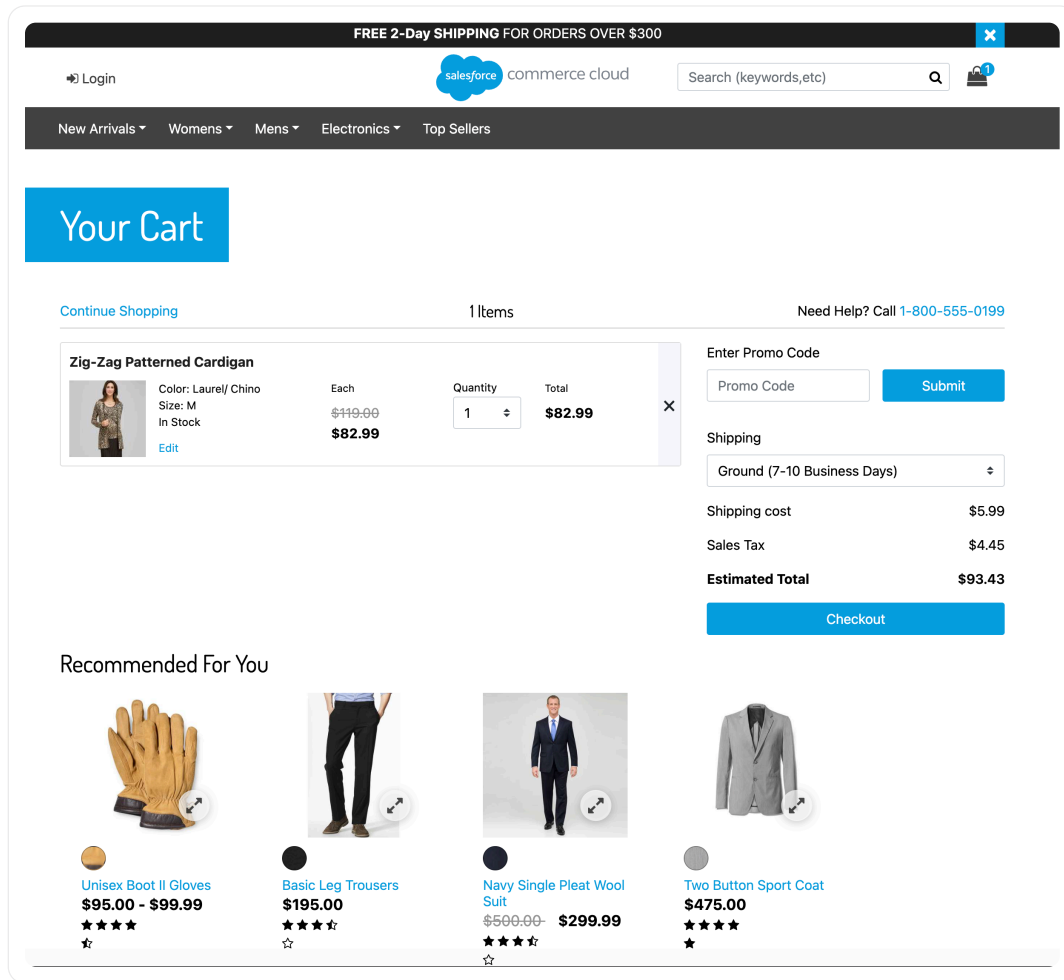
Template: * slots/personalization/home-scapi-compare



 Screenshot: Cart slot config showing the Template field

Template: * slots/personalization/cart

Template: * slots/personalization/cart-sfra



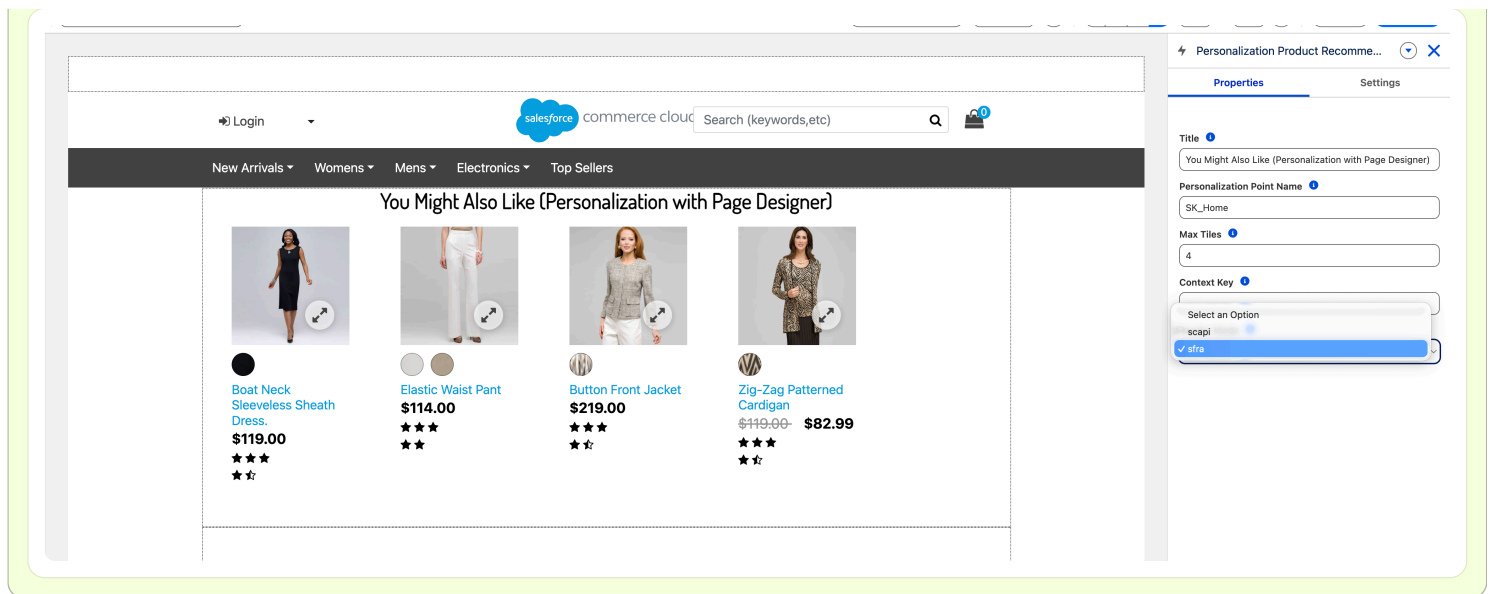
Alternative: Page Designer placement

Besides Content Slots, the same recommendation zone can be placed via Business Manager Page Designer: drag the "Personalization Product Recommendations" component (group: Personalization) onto a page/region, and set its Title / Personalization Point Name / Max Tiles / Context Key / Render Mode attributes per instance — Render Mode (scapi / sfra) is the Page Designer equivalent of swapping a Content Slot's Template field. Multiple instances on one page each resolve their own point name but share a single decision call.

Page Designer placement follows the same configuration pattern as Content Slots — set the component's Render Mode attribute to match your chosen hydration approach (scapi or sfra). See PageDesigner-Integration-Guide.md for a full walkthrough.

Page Designer screenshot

Page Structure panel with the "Personalization Product Recommendations" component nested in a region, and/or the Properties panel showing Title / Personalization Point Name / Max Tiles / Context Key / Render Mode filled in.



Part 4: Browser Verification (Console + Network)

Load a storefront page with a `scapi`-mode zone and open DevTools.

Step 1 — Console. Expect at most a `[PersonalizationTiles]` decision-API warning (the real SP decision call isn't wired in yet). Anything else means a config or CORS problem.

Step 2 — Network, config fetch. `PersonalizationTiles-GetConfig` → 200, `enabled: true`, all 5 required fields populated, response includes `getTokenUrl` and `pricesUrl`.

Step 3 — Network, token. `PersonalizationTiles-GetToken` → 200. Same-origin call to this cartridge's own controller (not a direct browser call to SLAS) — a failure here points at `ps_scapiClientId` / `ps_scapiRedirectUri` or the SLAS client itself.

Step 4 — Network, products. SCAPI Shopper Products GET → 200, tiles render with live price/image data.

Step 5 — Network, prices. `PersonalizationTiles-GetPrices` → 200; a price-book-markdown SKU shows a strike-through list price next to the sale price.

SCAPI Shopper Products only ever returns one resolved price per product — no list/sale distinction to expose. `GetPrices` closes that gap without calling SCAPi again: it's a server-side-only endpoint that reuses `Tile-Show`'s own pricing codepath (`ProductSearchModel` → `searchPrice` decorator → `promotionCache`) to produce byte-identical strike-through markup, which the client applies on top of the SCAPi-hydrated tile.

This is a **price-book markdown** fix, not a promotion-discount fix. A *true* promotion-discounted price (e.g. "10% off" applied at checkout, not baked into a price book) is still not obtainable from SCAPi Shopper Products — still open.

Step 6 — DOM check, distinguishing `scapi` vs `sfra` zones. Inspect each zone's `.ps-zone` container's `data-ps-render-mode` attribute:

```
<div class="ps-zone" data-ps-render-mode="sfra" ...> <!-- native Tile-Show tiles -->
<div class="ps-zone" data-ps-render-mode="scapi" ...> <!-- SCAPi-hydrated tiles -->
```

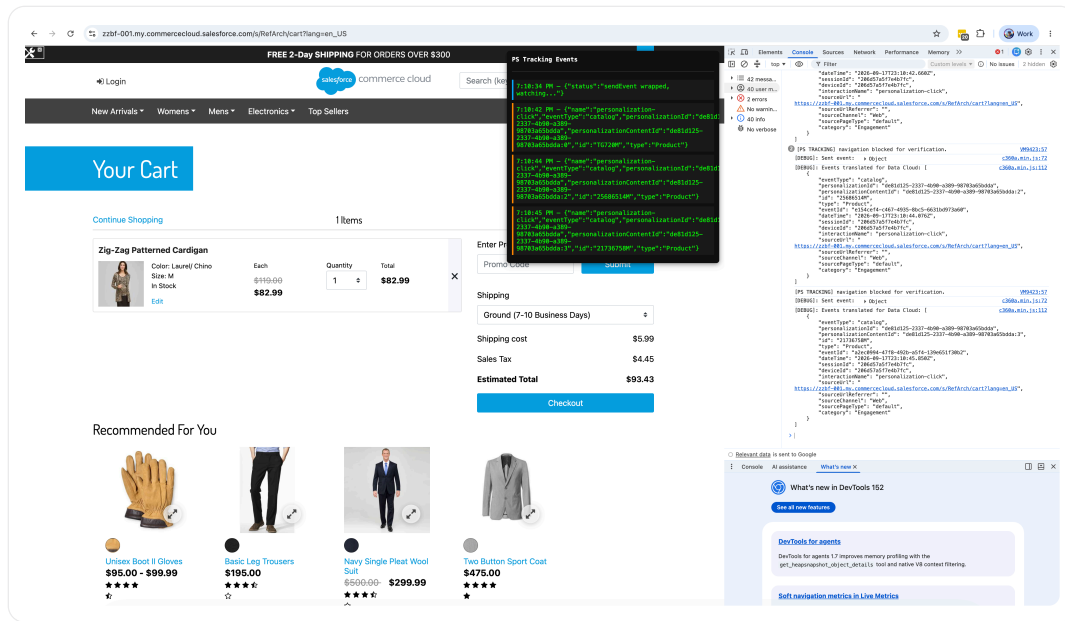
Step 7 — Strongest proof. Log in as a customer with group pricing or an assigned promotion, reload: the `sfra` zone's prices update to reflect the logged-in shopper; the `scapi` zone stays on guest-only pricing regardless of login state.

Step 8 — Activity tracking (`personalization-view` / `personalization-click`). Requires `ps_activityTrackingEnabled` = true and a working `ps_dcSdkUrl` (Data Cloud beacon SDK, `c360a.min.js`) — both from Part 2. `ps_dcSdkUrl` is tenant-specific and provisioned by Data Cloud, not invented per install — confirm it with Data Cloud before go-live if activity tracking is in scope. Paste `cartridge/docs/verify-activity-tracking.console.js` into DevTools Console **immediately** after loading the page (ideally before tiles render, so the `personalization-view` burst is caught too, not just clicks).

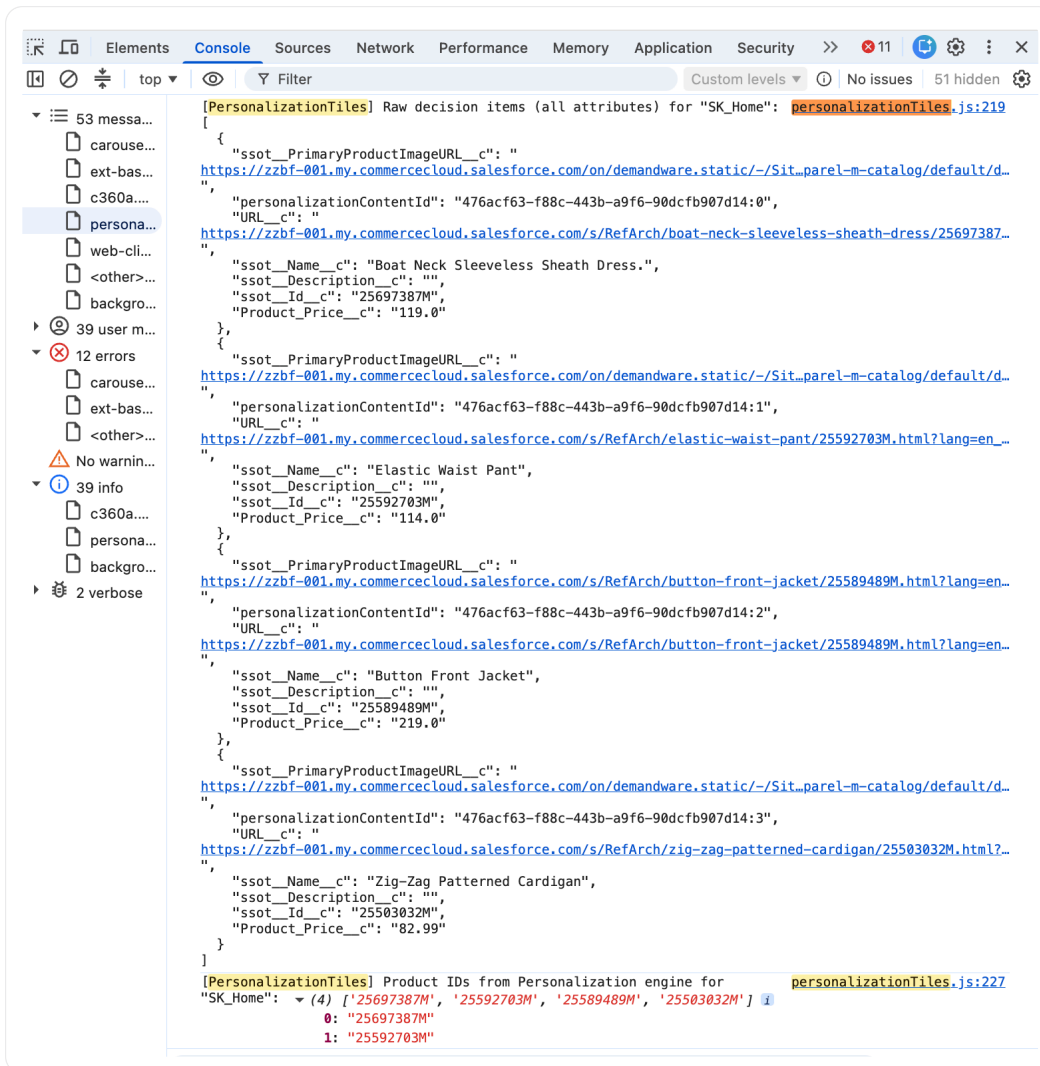
Q: What is this panel actually doing? It's a **Salesforce-provisioned verification utility** — supplied as part of this cartridge's tooling, not an out-of-the-box Salesforce/Data Cloud product feature. It's a throwaway DevTools console script that intercepts tile click events (blocking only their default PDP navigation, without stopping propagation, so the tile's real click handler still fires normally) and displays every `personalization-view` / `personalization-click` event live, as it's fired, in a floating panel.

- A floating "PS Tracking Events" panel pinned top-right of the page. Blue-bordered rows = `personalization-view`, orange-bordered rows = `personalization-click`.
- Console `[DEBUG]` lines from `c360a.min.js`: `Sent event` followed by `Events translated for Data Cloud: [...]`, showing the translated payload — `eventType`: `"catalog"`, `personalizationId`, `personalizationContentId`, product `id` / `type`, `sessionId`, `deviceId`, `interactionName`, `sourceUrl`, `sourceChannel`: `"Web"`, `sourcePageType`, `category`: `"Engagement"`.
- Clicking a tile also logs `[PS TRACKING] navigation blocked for verification.` in the console, confirming the click-block described above — while the tile's own click handler (and its real `sendEvent()` call) still fires normally.

🖥️ Screenshot: "PS Tracking Events" panel + console DEBUG logs showing translated Data Cloud events



🖥️ Screenshot: DevTools Console tab on a loaded storefront page



🖥️ Screenshot: DevTools Network tab showing GetConfig → GetToken → SCAPI products → GetPrices

Name	Status	Type	Initiator	Size	TL...
personalizationTiles.js	200	script	home?lang=en_US:793	11.5 kB	8...
PersonalizationTiles-GetConfig	200	fetch	personalizationTiles.js:38	0.7 kB	2...
PersonalizationTiles-GetPrices?ids=25697387M,25592703M,25589489M,25503032M	200	fetch	personalizationTiles.js:38	0.5 kB	6...
PersonalizationTiles-GetProducts?ids=25697387M,25592703M,25589489M,25503032M	200	fetch	personalizationTiles.js:38	2.9 kB	6...

Part 5: SLAS, Account Manager & CORS Preferences Setup

The `scapi` render mode needs two separate things from Account Manager: (1) a SLAS **public (PKCE)** client scoped to Shopper Products, and (2) that storefront's domain registered against SCAPI's **CORS Preferences** admin API — a completely separate surface from the SLAS client itself (there is no CORS allow-list field on a SLAS client). CORS Preferences is configured per SLAS `client_id` + site, typically via a *different* Account Manager API client than the shopper-facing SLAS client.

1. In Account Manager, create or confirm an existing SLAS public (PKCE) client for the org, scoped to Shopper Products.
2. Register the storefront domain(s) against SCAPI CORS Preferences for that `client_id` + site: `PUT .../configuration/cors/v1/organizations/{orgId}/cors?siteId={site}`, using a *separate* Account Manager API client with the `SALESFORCE_COMMERCE_API` role and `sfcc.cors-preferences.rw` scope. **The token requested for this call must explicitly include the scope `SALESFORCE_COMMERCE_API:<tenant-id>`** (e.g. `SALESFORCE_COMMERCE_API:zzbf_001`), in addition to `sfcc.cors-preferences.rw` — omitting it produces a generic `403 Forbidden` that looks identical to a real entitlement gap, even when the client's role/scope/tenant-filter config is otherwise correct. This was a real multi-week blocker on this project before the scope requirement was identified — check this first if CORS Preferences 403s unexpectedly.
3. Register a `redirect_uri` on the SLAS client (any value — SLAS's guest PKCE exchange never delivers a response there, it just has to be registered) and record it into `ps_scapiRedirectUri`.
4. Record the SLAS client id into `ps_scapiClientId`.

⚠ **Two separate Account Manager grants are needed here, not one.** (1) Creating/confirming the SLAS client (via the Account Manager UI or the `b2c` CLI) requires SLAS Admin authorization on that specific org. (2) Registering CORS Preferences (Step 2 above) requires a *different* AM API client with the `SALESFORCE_COMMERCE_API` role — and its token request must explicitly include the `SALESFORCE_COMMERCE_API:<tenant-id>` scope or every SCAPI Admin API call returns a generic 403 that looks like a missing entitlement. Both are separate grants from standard Business Manager access. Request both early in the rollout, alongside the Prerequisites checklist above, so neither surfaces as a late-stage blocker. Validate the SLAS client and CORS Preferences registration in a lower environment (Staging/UAT) before relying on `scapi` mode in production.

■ **Optional: inspect SLAS clients via CLI.** If you have the `b2c` CLI configured, you can confirm a client's existing config without the Account Manager UI:

```
b2c slas client list --tenant <tenant> --short-code <short-code>
b2c slas client get --tenant <tenant> --short-code <short-code> <client-id>
```

Part 6: Rollback / Disable

- **Config-only issue** (wrong short code, client id, etc.): fix the Site Preference value in Business Manager — no code redeploy needed, takes effect on next page load since `GetConfig` is uncached.
- **Code issue:** revert to the previously active code version in Business Manager → Administration → Site Development → Code, or remove `plugin_b2c_d360_personalization` from the Cartridge Path to fully disable the feature storefront-wide without a code rollback.
- **Single placement:** unpublish/delete the Page Designer page, or reassign the Content Slot to a different template — no code or Site Preference changes needed.
- No data migrations or irreversible state are touched by this cartridge, so rollback is low-risk.

Part 7: Current Limitations

- **Decision API is a stub** — `getPersonalizationDecisions()` isn't yet wired to the real Personalization decision/zone API; expect a `[PersonalizationTiles]` console warning until that's done.
- **Tile markup is placeholder** in `scapi` mode until your storefront's real product-tile markup is swapped in.
- **No built-in CI/CD pipeline** — deploy this cartridge via your standard code-deployment process (WebDAV, Business Manager upload, or your existing CI/CD pipeline); none is bundled.
- **Promotion-discounted strike-through pricing is not yet supported** — `GetPrices` (Part 4, Step 5) covers price-book markdowns only; a true promotion-discounted price isn't obtainable from SCAPI Shopper Products today.